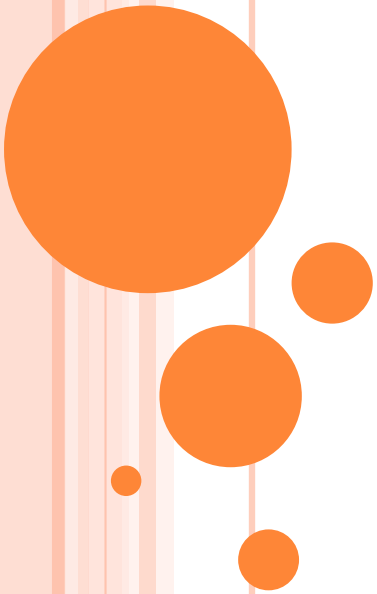


Fault Tolerance – Reliable Group Communication

Credit for slides to:

Timofei Istomin

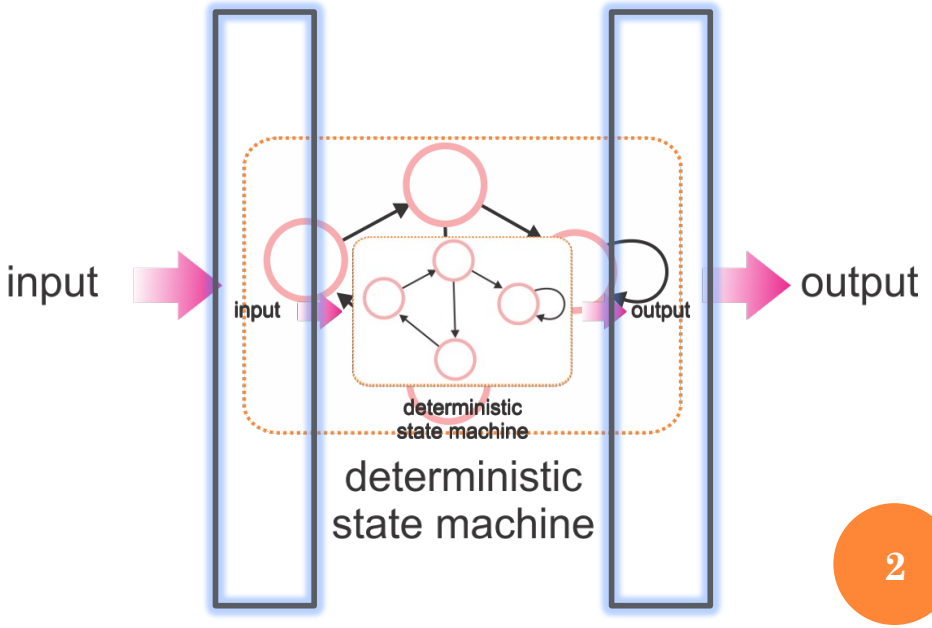
Giuliano Mega

Davide Frey

Luca Mottola

Reliable group communication

- It is of critical importance if we want to exploit process resilience by replication
- Classical example: the *state machine* approach;
  - make your process a *deterministic state machine*, so that…
  - … given *n* copies of your process, if you feed them the same inputs:
    1. they will emit the same outputs
    2. their internal state will be the same



Reliable group communication

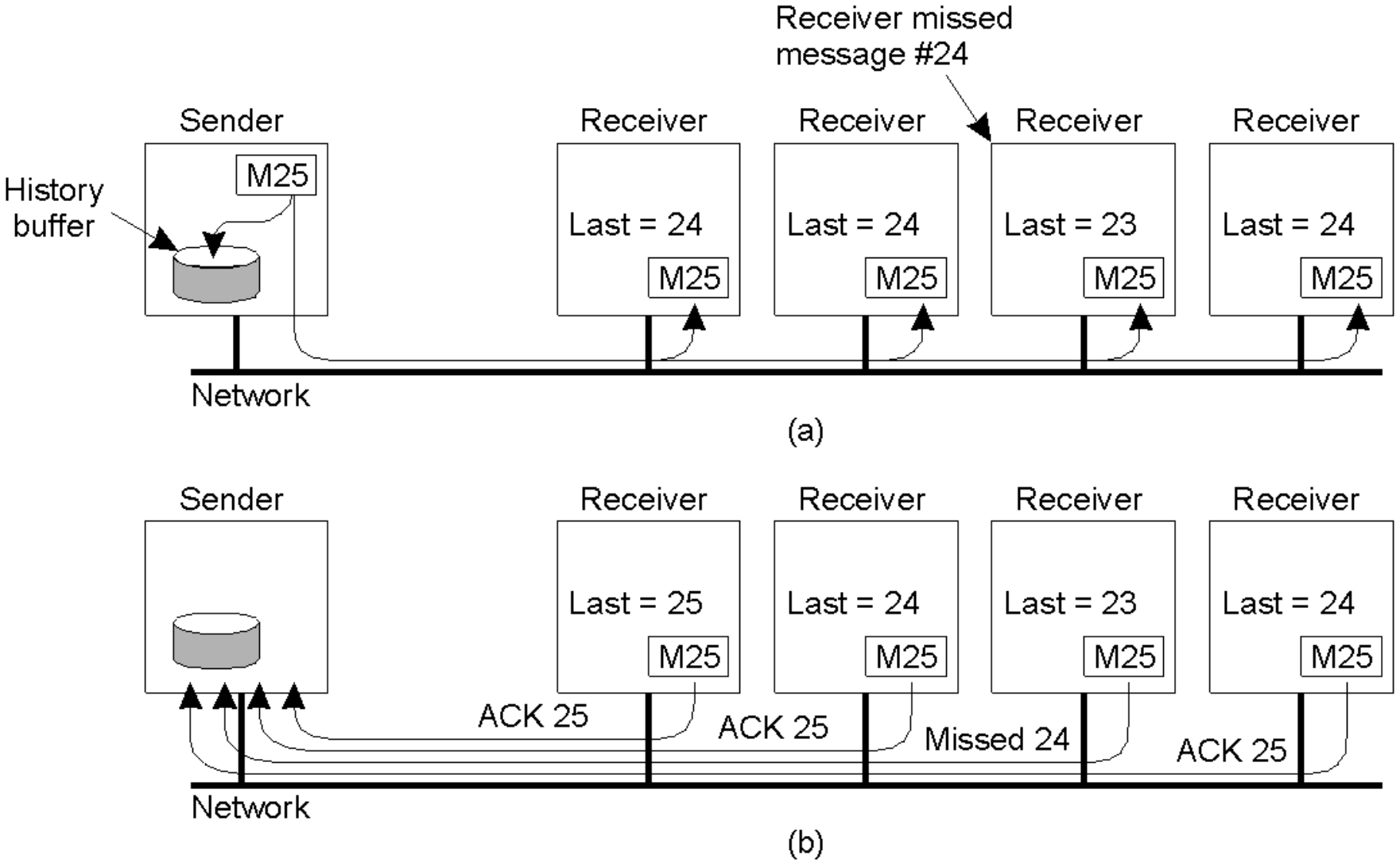
- Reliable multicast is surprisingly tricky
  - Yet, we need it to deliver messages to all members in a process group
- Usually, all we have is a reliable point-to-point service or an unreliable multicast service
  - Achieving reliable multicast through multiple reliable point-to-point channels may not be efficient
- Plus…
  - … what happens if the sender crashes while sending?
  - What if the group changes while a message is being sent? Who should get the message, and who should not?

Reliable group communication

- We look at two cases for reliable multicast
- **Case 1:** groups are fixed (i.e., processes do not crash), communication failures only
  - All group members should receive the multicast
  - Unreliable multicast primitive is available
- **Case 2:** faulty processes, but reliable channels

- We start from **Case 1**

Basic Reliable Multicast

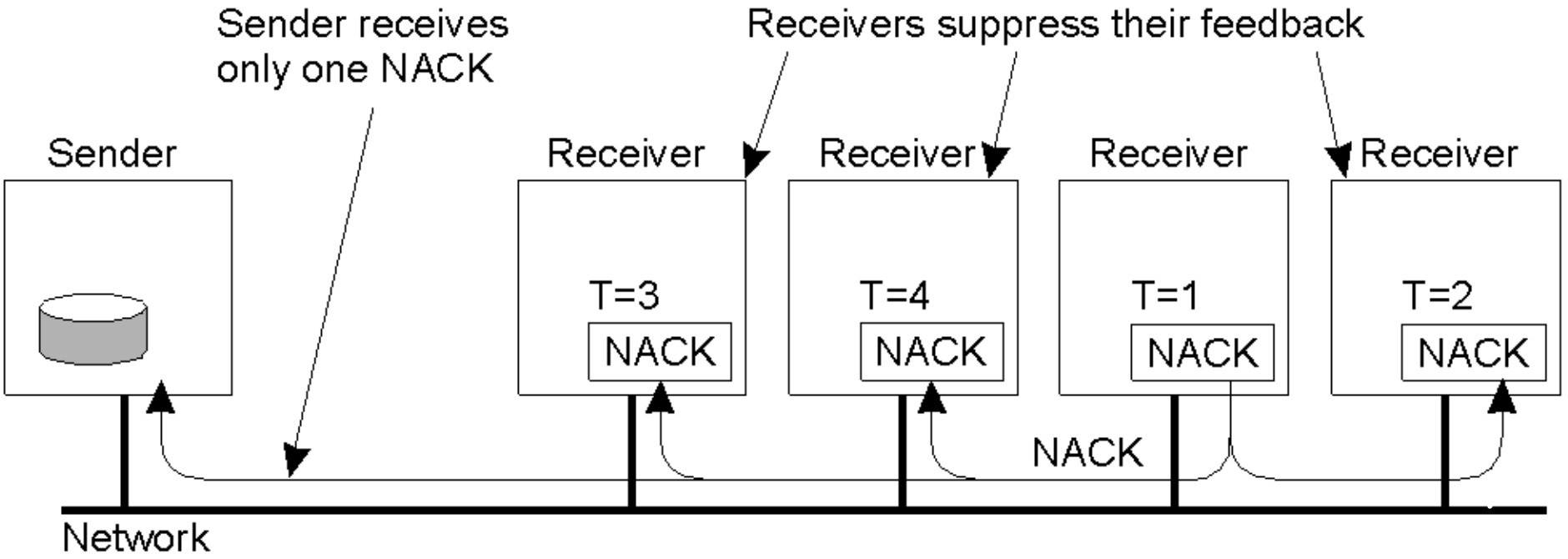


Basic Reliable Multicast

- What if we make the receivers ACK all messages?
- Key problem: too many ACKs:
  - "feedback implosion"
- Mitigation strategies:
  - piggyback ACK messages on traffic (only works if traffic is constant enough), and
  - retransmit using point-to-point primitive
- Scalability problems still persist
- Negative ACKs: only send feedback if message not received
  - sender might have to cache messages forever
  - still may result in too many ACKs

Scalable Reliable Multicast (SRM)

- A first solution is *non-hierarchical feedback control*, implemented in SRM
  - Negative acknowledgements are multicast
  - Randomly staggered in time
  - Nodes that receive someone else's NACK suppress theirs
- Used in practice in several applications

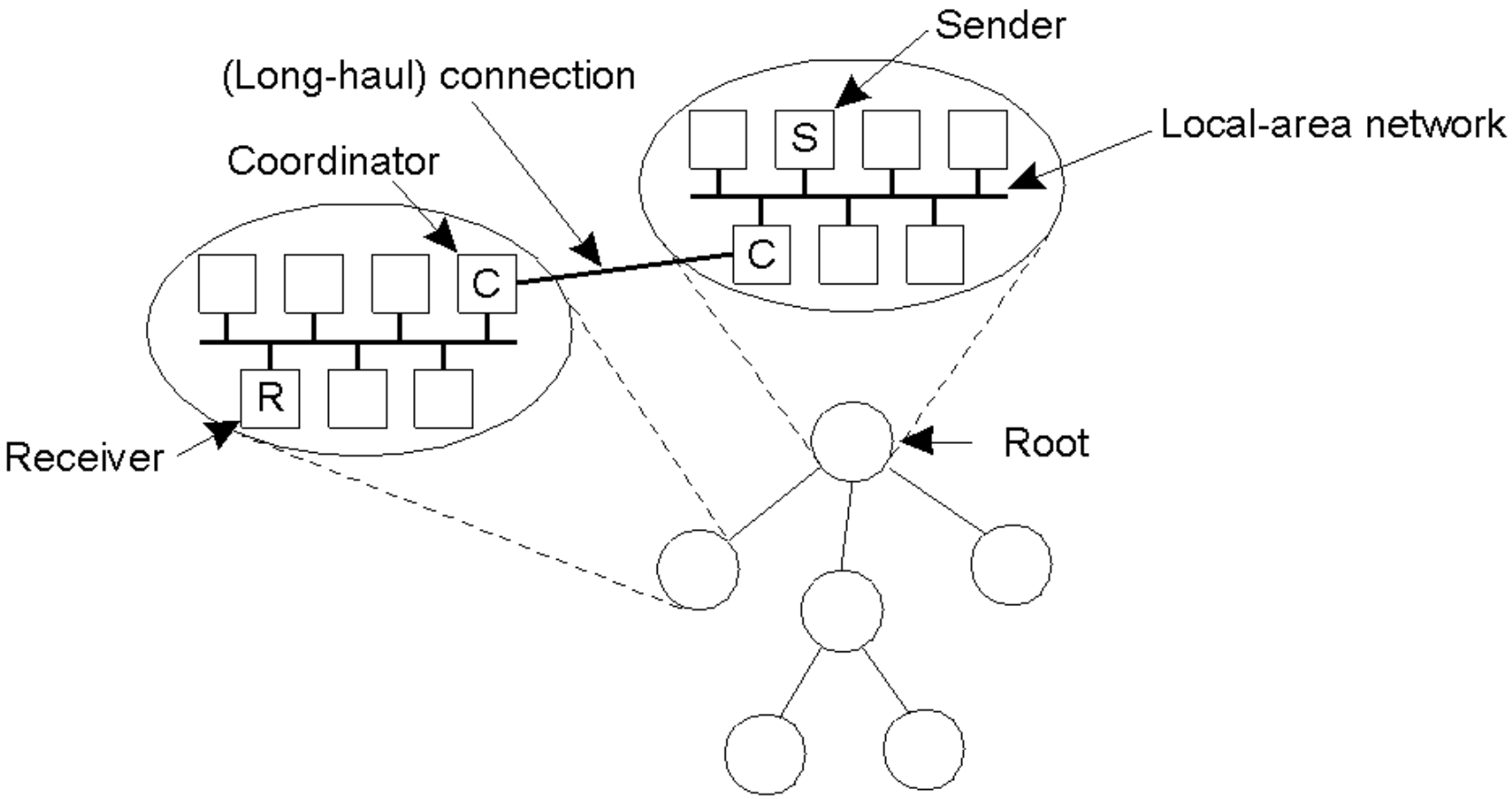


Scalable Reliable Multicast

- Staggering "the right amount" is non-trivial
  - depends on network delays, right value might be group-dependent, especially over WANs
- Processes that don't lose messages still receive NACKs from those who do
  - Dynamically create separate groups for processes that tend to miss messages together, difficult to achieve in large-scale settings
- Ultimately, flat groups will always have scalability limitations

Hierarchical Feedback Control

- With Hierarchical Feedback control, receivers are organized in groups headed by a coordinator



Hierarchical Feedback Control

- The coordinator can adopt any strategy within its group
- In addition, it can request retransmissions to its parent coordinator
  - A coordinator can remove a message from its buffer if it has received an ACK from
    - All the receivers in its group
    - All of its child coordinators
- The problem is that the hierarchy must be constructed and maintained
  - Difficult to implement on physical networks, usually done via application-level overlays

Reliable Multicast

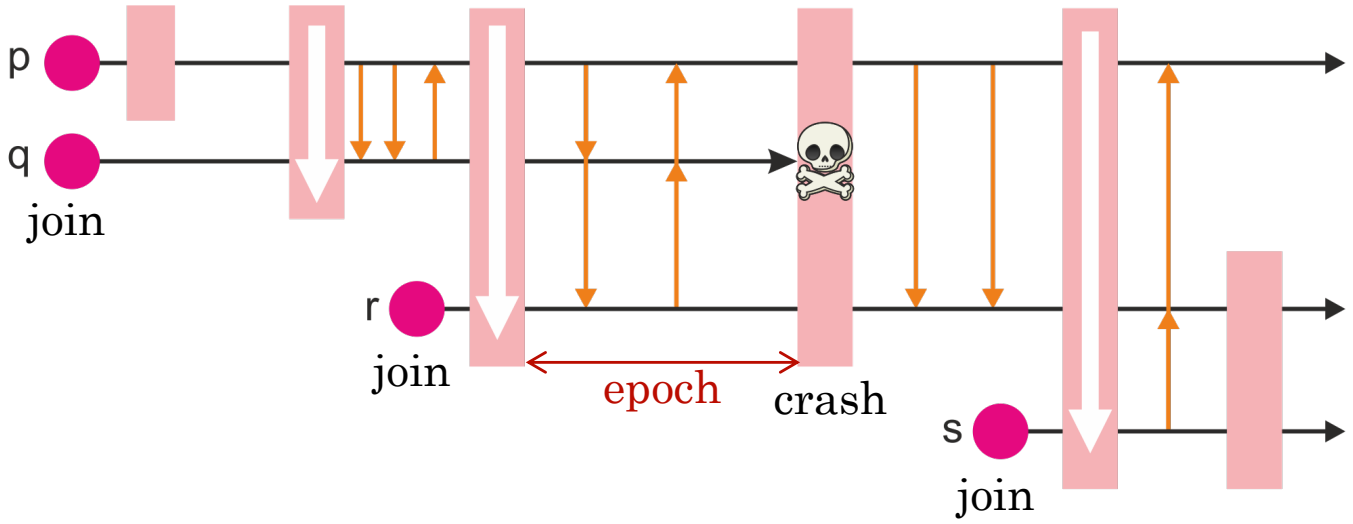
- Recall – two cases:
  - **Case 1:** fixed groups, unreliable channels;
  - **Case 2:** dynamic groups (processes can join, leave or crash), reliable channels

- Now we look at **Case 2**

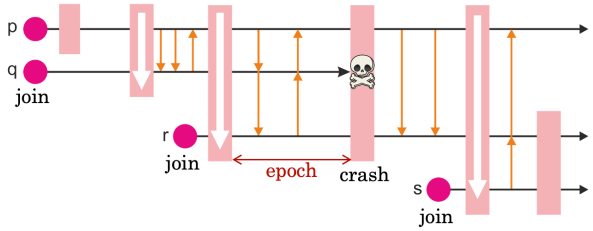
- The main problem is **dynamic membership**:
  - group membership may change while multicasts are being issued: ***who should get the message***?
  - to simplify the application programmer's life the middleware can provide **virtual synchrony (VS)**

Close Synchrony vs Virtual Synchrony

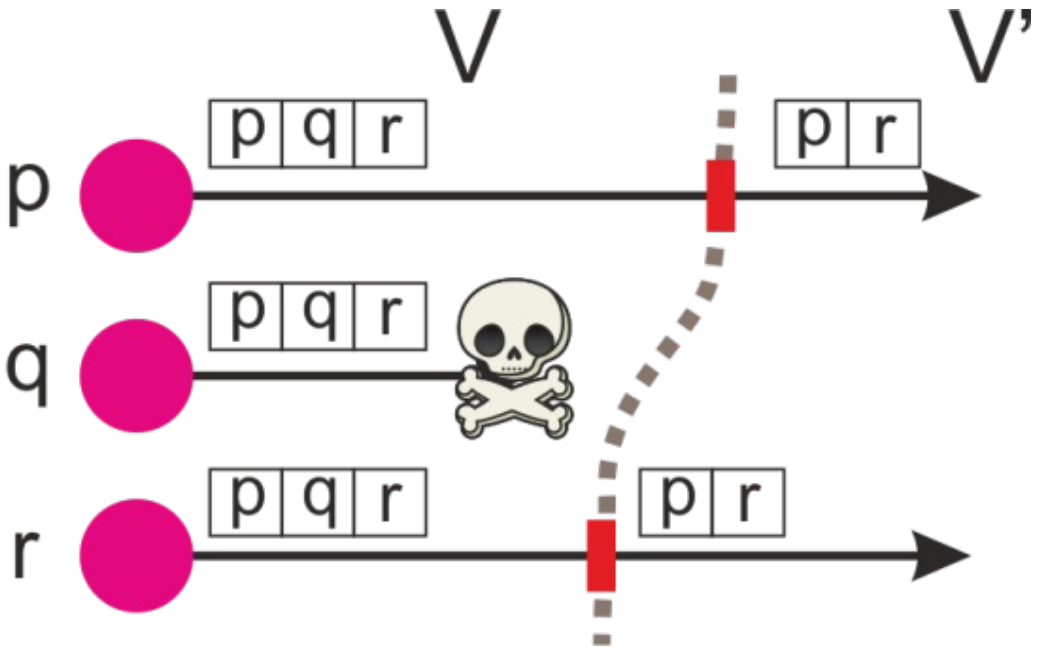
- Virtual synchrony *emulates* close synchrony where the key events (multicasts, group changes) are *instantaneous*
- Easy to reason about, since crashes or joins never overlap with message delivery
- Recall that multicasts are reliable (by assumption): always get to everyone in the group



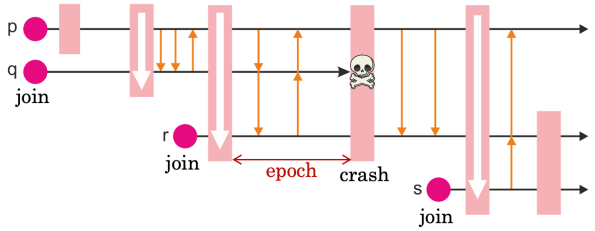
Views and View Synchrony



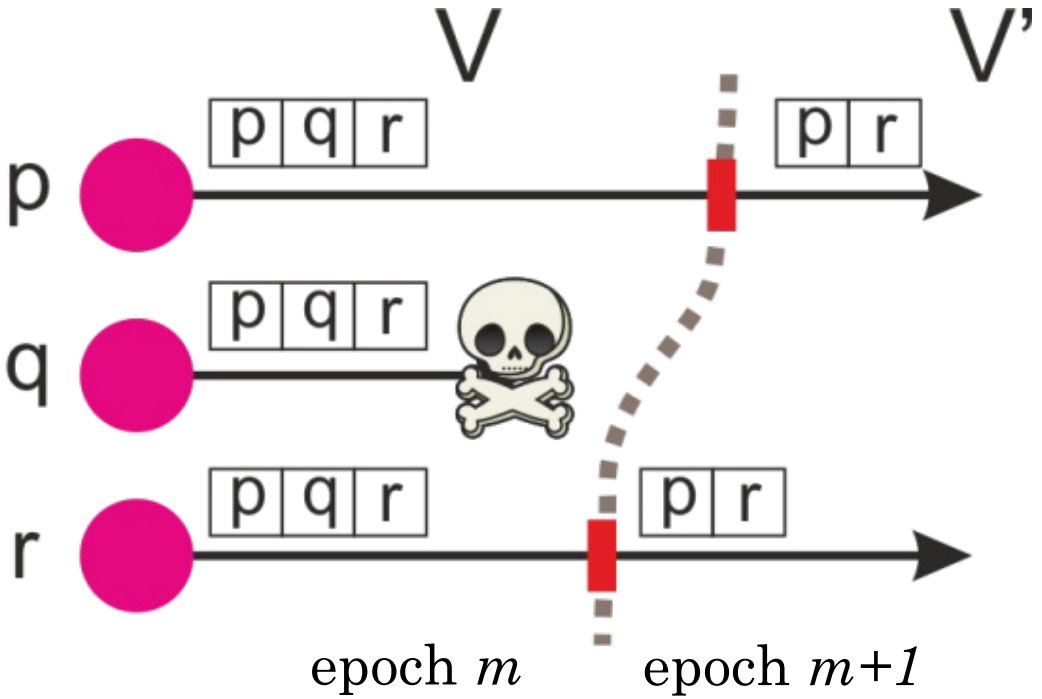
- In VS, each process maintains a "view" of the group
  - i.e., local knowledge of who is part of the group, **and therefore to whom messages should be delivered to**
- Views "get replaced" as the group changes: processes *install new views* when a new member joins the group or someone leaves the group



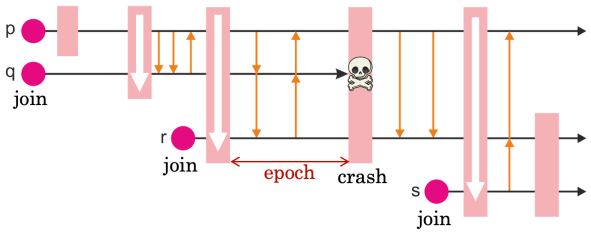
Views and View Synchrony



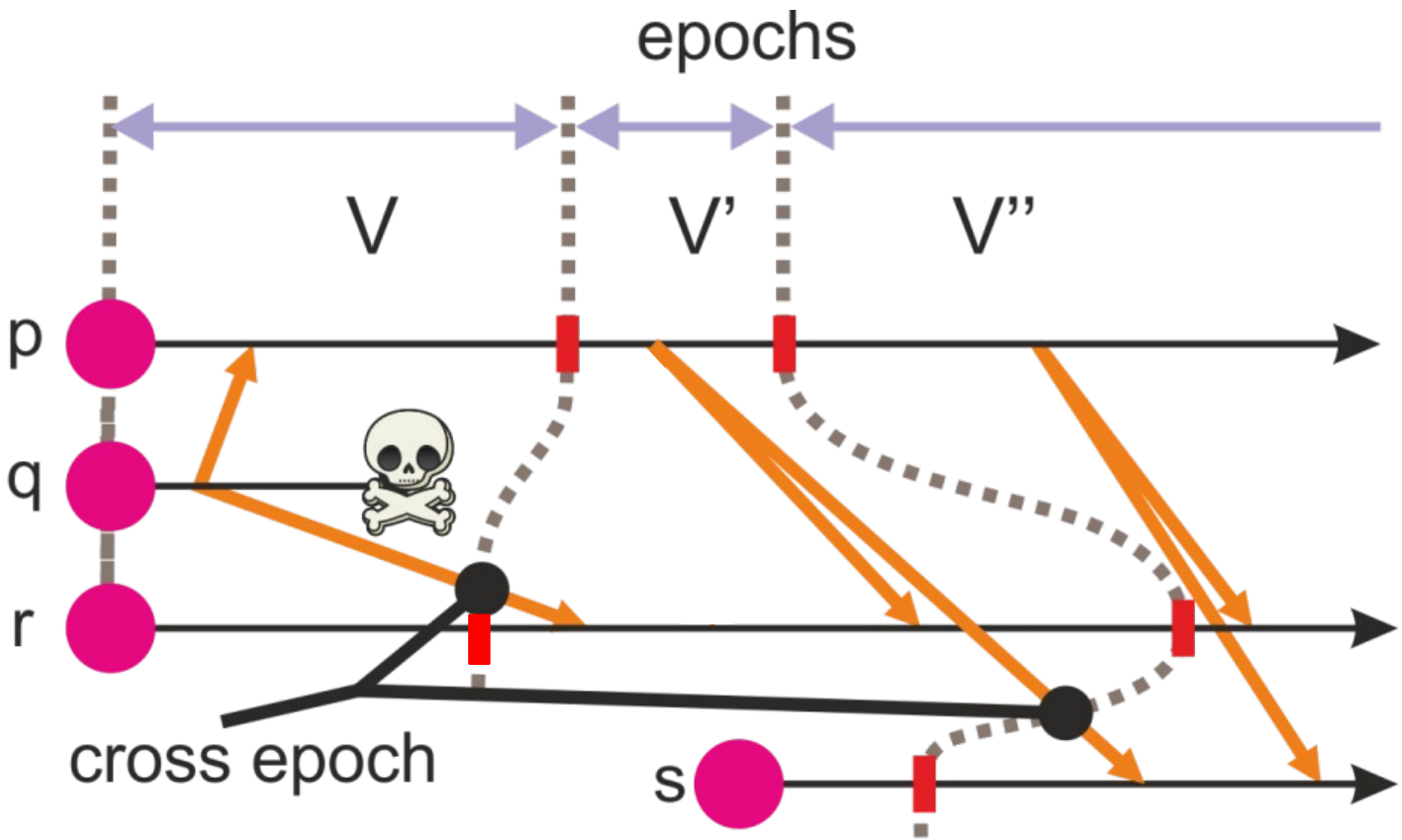
- The system guarantees (under some assumptions) **that all correct (i.e., non-crashed) processes will see the *same* sequence of group views**
- Therefore, each view defines a global **epoch**



Views and View Synchrony



- Another important property guaranteed by virtual synchrony: **multicasts do not cross epoch boundaries**

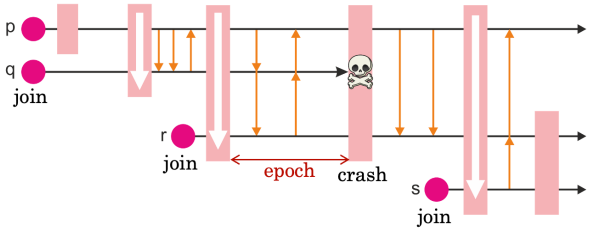


V={p,q,r}

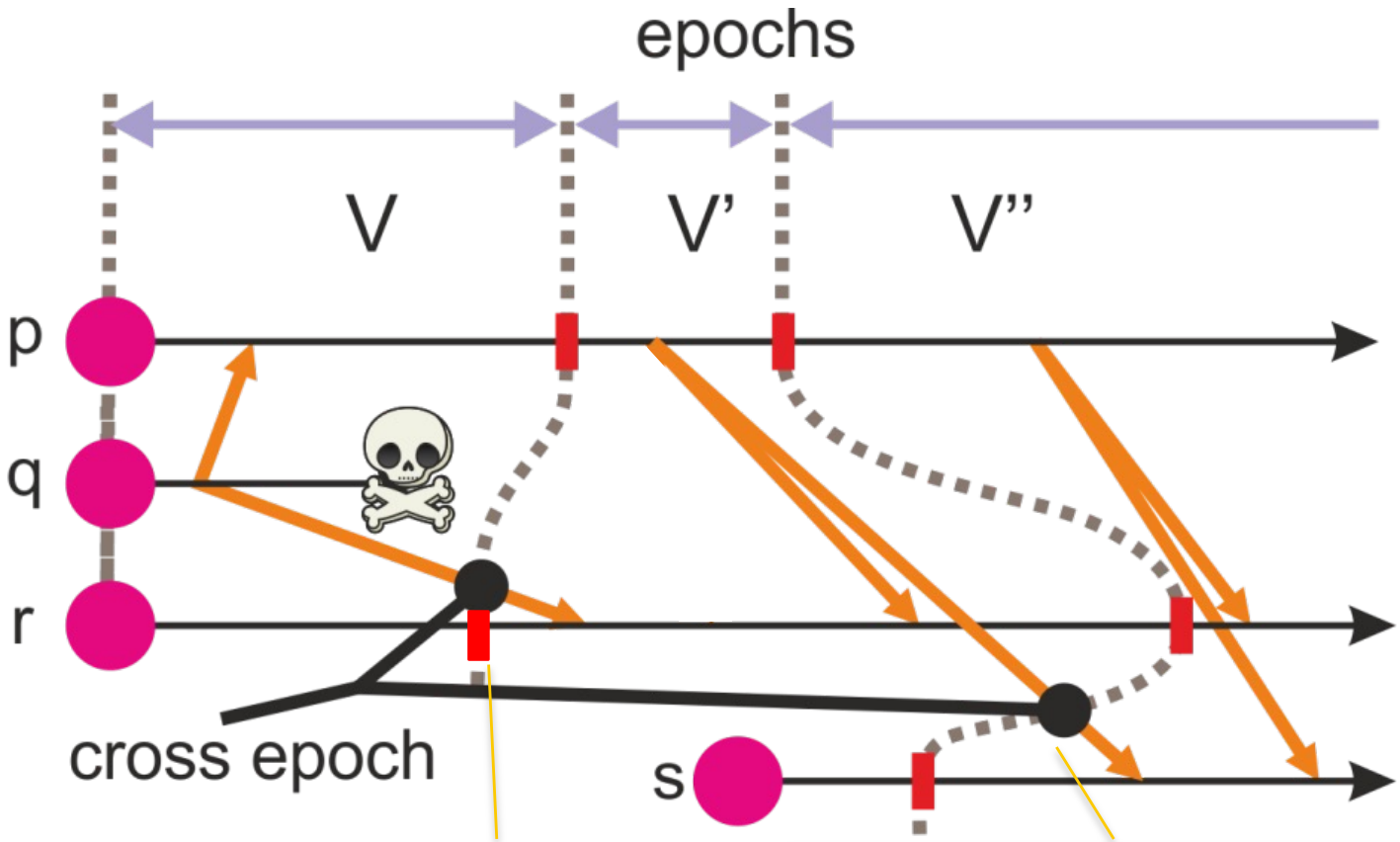
V'={p,r}

V''={p,r,s}

Views and View Synchrony



- Why crossing epoch boundaries is a problem?



V={p,q,r}

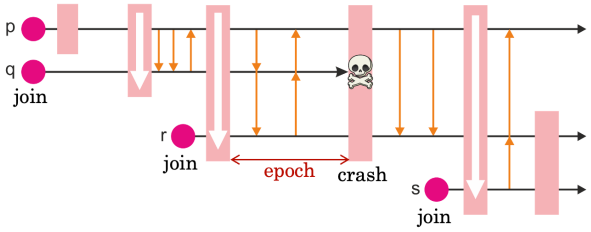
V'={p,r}

V''={p,r,s}

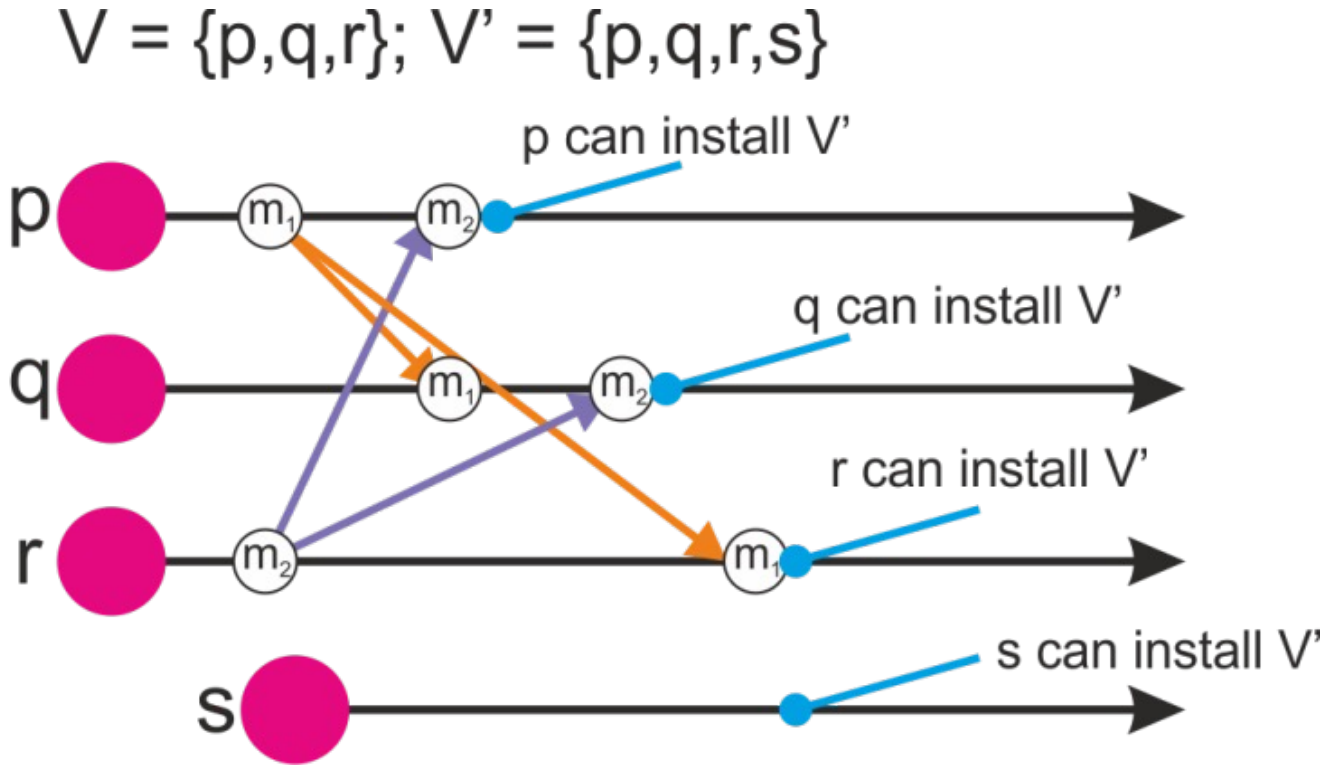
R does not have the same information as P does when it gets informed about Q's crash

P and R are not sure what information S have received since it has joined

Views and View Synchrony



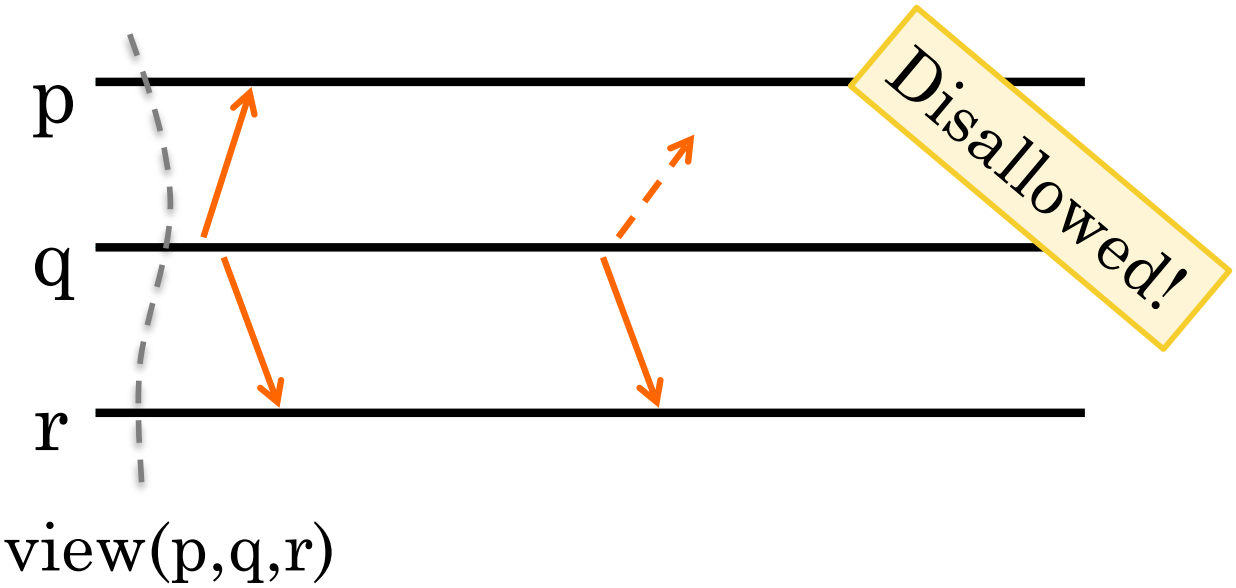
- *"Multicasts do not cross epoch boundaries"*
  - If there are multicasts for messages m_1 and m_2, and a view change occurs …
  - … to satisfy view synchrony, these multicasts **must complete** before the new view is installed

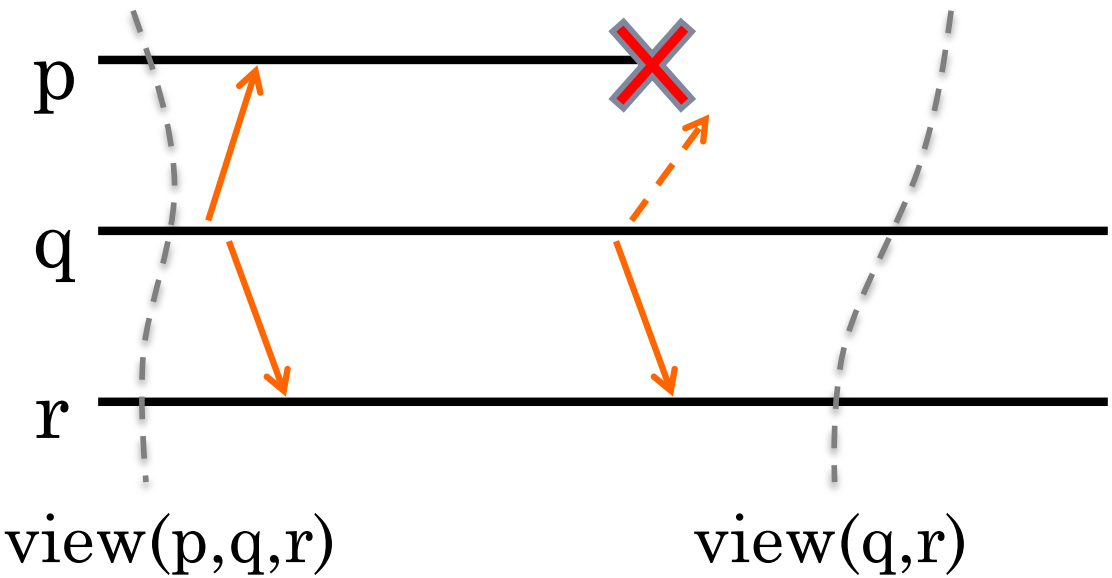


Multicasts in Virtual Synchrony

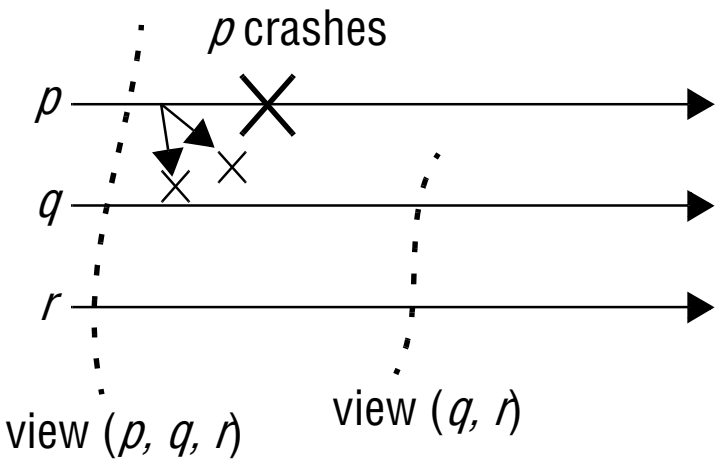
- What does it mean for multicast to "complete"? Who has to get it or not get it?
- Virtual synchrony multicasts are **reliable**
- A message multicast in an epoch E defined by view V should either:
  - be **delivered** within E by all **operational** participants
  - be **delivered** by none of the **operational** participants
  - One of these conditions is met: multicast is complete
- To exclude trivial protocols (just discard everything), the *"or none"* case is only allowed if the sender crashes
  - All "evidence" erased: as if it had never been sent
  - We only care about operational participants…

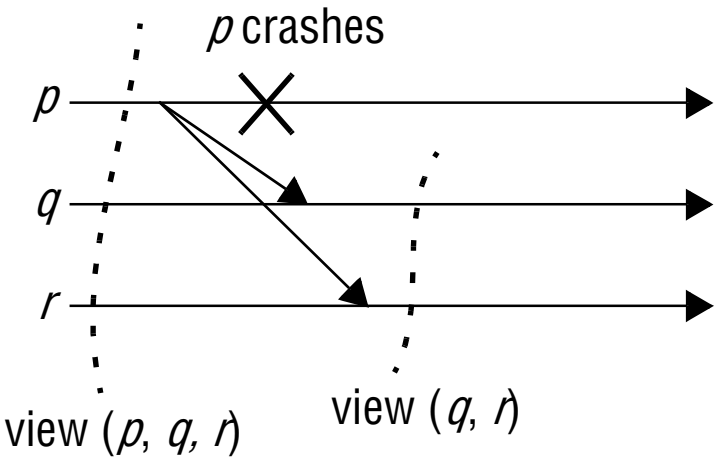
Examples

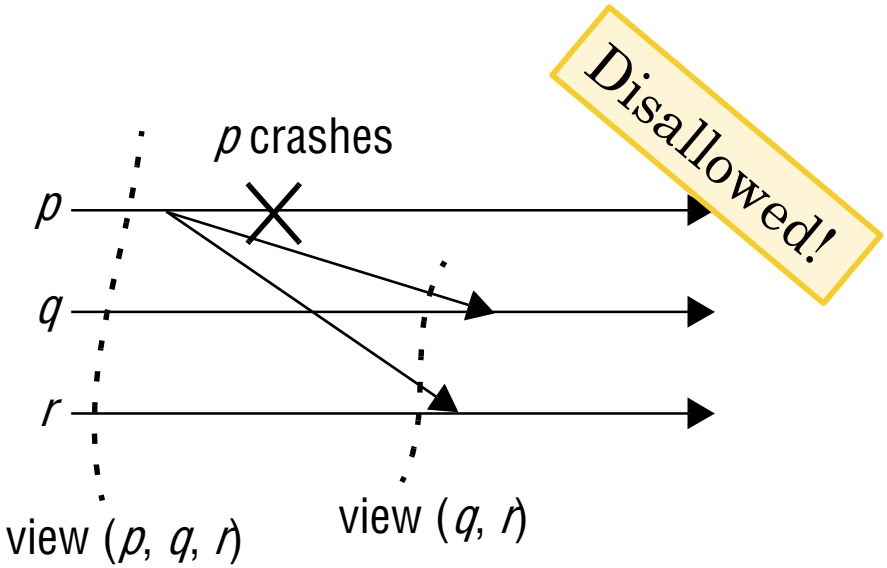


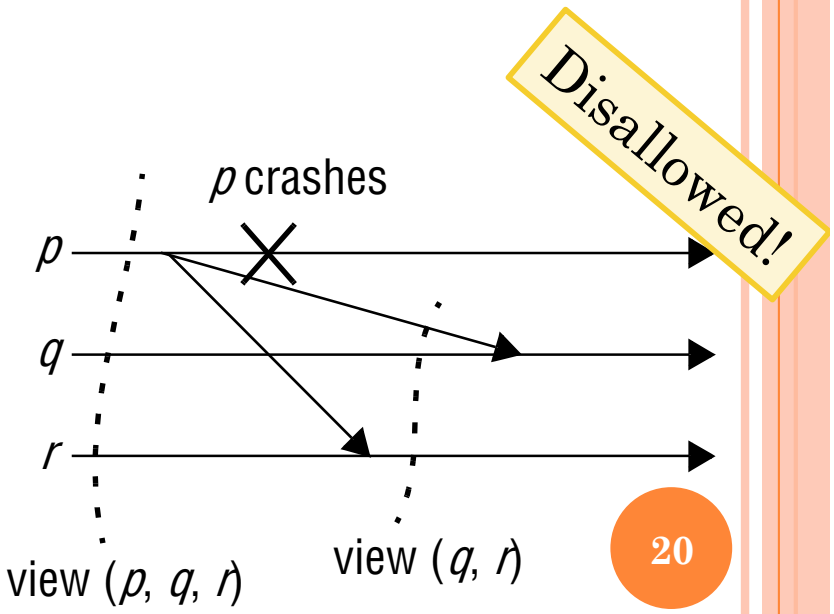


EXAMPLES


p crashes
p
q
r
view (p, q, r)
view (q, r)


p crashes
p
q
r
view (p, q, r)
view (q, r)


Disallowed!
p crashes
p
q
r
view (p, q, r)
view (q, r)


Disallowed!
p crashes
p
q
r
view (p, q, r)
view (q, r)

The Flavors of Virtual Synchrony

- In virtual synchrony, messages can't cross epochs …
- … but what can happen inside an epoch?
- Three base variants w.r.t. message ordering:
  - unordered;
  - FIFO ordered;
  - causally ordered;
- … where each of those can be:
  - totally ordered;
  - not totally ordered;
- … for a total of six variants

Detour: slides about message ordering…

Naming the Flavors

- The six versions of virtually synchronous reliable multicast

Multicast	Basic Message Ordering	Totally-ordered Delivery?
Reliable multicast	None	No
FIFO multicast	FIFO-ordered delivery	No
Causal multicast	Causally-ordered delivery	No
Atomic multicast	None	Yes
FIFO atomic multicast	FIFO-ordered delivery	Yes
Causal atomic multicast	Causally-ordered delivery	Yes

Exercise 1

Is this execution virtually synchronous? What about FIFO, causally, or total ordered? Assume all start sharing the same view V0.

P1

1. delivers M1
2. installs view V1
3. delivers M2
4. multicasts M3
5. delivers M3
6. installs view V2

P2

1. multicasts M1
2. delivers M1
3. installs view V1
4. multicasts M2
5. delivers M2
6. delivers M3
7. installs view V2

P3

1. delivers M1
2. installs view V1
3. delivers M3
4. delivers M2
5. installs view V2

P4

1. installs view V2
2. delivers M2

Group views
V0={P1,P2,P3,P4}
V1={P1,P2,P3}
V2={P1,P2,P3,P4}

Exercise 2

Is this execution virtually synchronous? What about FIFO, causally, or total ordered? Assume all start sharing the same view V0.

P1

1. multicasts M1
2. delivers M1
3. installs view V1
4. delivers M2
5. installs view V2
6. delivers M3

P2

1. delivers M1
2. installs view V1
3. delivers M2
4. installs view V2
5. delivers M3

P3

1. installs view V2
2. multicasts M3
3. delivers M2
4. delivers M3

P4

1. delivers M1
2. installs view V1
3. multicasts M2
4. delivers M2
5. installs view V2
6. delivers M3

Group views
V0={P1,P2,P3,P4}
V1={P1,P2,P4}
V2={P1,P2,P3,P4}